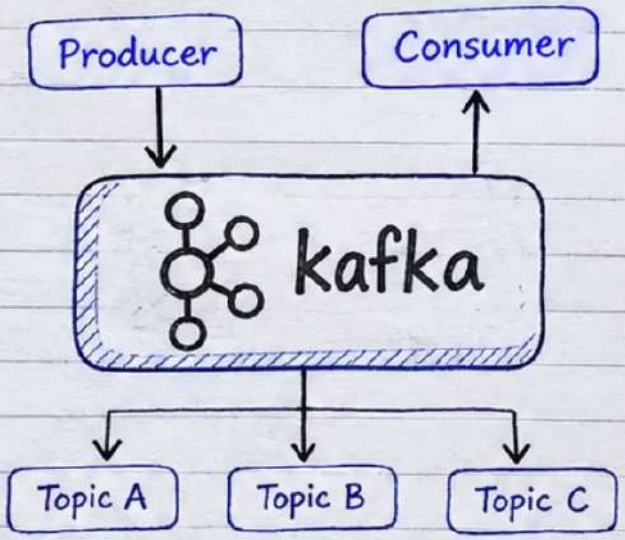
Apache Kafka

From Zero to FAANG

Complete Handwritten Study Handbook

15 Pages

- ☑ Kafka Fundamentals
- ☑ Architecture & Core Components
- ☑ Producers & Consumers
- ☑ Delivery Semantics
- ☑ Replication & Fault Tolerance
- ☑ Kafka with Java & Spring Boot
- ☑ Performance & Optimization
- ☑ Real-World Use Cases
- ☑ FAANG Interview Questions
- ☑ System Design & Best Practices



SUBTOPIC : Introduction to Apache Kafka

1. What is Apache Kafka ?

- Apache Kafka is a distributed, open-source **event streaming platform** used to build real-time data pipelines and streaming apps.
- It can publish, store, and process large amounts of data in a **fault-tolerant** and **scalable** way.

Key Terms

- **Event** : A piece of data / record.
- **Streaming** : Continuous flow of events.
- **Pipeline** : System to move data from source → processing → sink.
- **Distributed** : Runs across multiple machines (cluster).

2. Why was Kafka Created ?

- To handle massive, real-time data (e.g. user activity, logs, payments, etc.)
- To **decouple** data producers and consumers.
- To provide **high throughput**, **low latency** and scalability.
- To build a unified, reliable event streaming system for modern distributed applications.

3. Traditional Messaging vs Kafka

Feature	Traditional MQ	Apache Kafka
Architecture	Queue based	Log (event stream)
Throughput	Lower	Very High
Scalability	Limited	Highly scalable
Data Retention	Short (consumed and removed)	Long (configurable)
Use Case	Simple messaging	Real-time streaming, analytics, big data

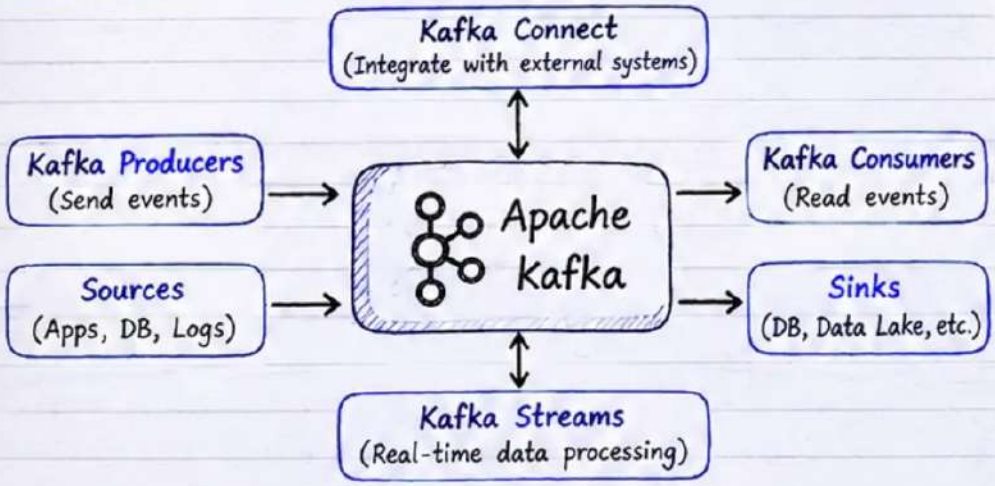
4. Real-World Use Cases

- ☑ Real-time log collection (e.g., from microservices)
- ☑ User activity tracking (e.g., clicks, views, likes)
- ☑ Payment & order processing (e.g., e-commerce)
- ☑ Real-time analytics (e.g., dashboards, fraud detection)
- ☑ Data integration between different systems
- ☑ Event-driven microservices communication

★ EXAM POINT

- Kafka is a distributed, fault-tolerant, and highly scalable event streaming platform.
- It is used for real-time data processing and building event-driven systems.

5. Kafka Ecosystem Overview



Other Components :

- **Kafka Connect** : For data integration (source/sink).
- **Kafka Streams** : For real-time stream processing.
- **Schema Registry** : Manages data schemas (e.g., Avro).
- **ZooKeeper / KRaft** : For cluster management (older/new).

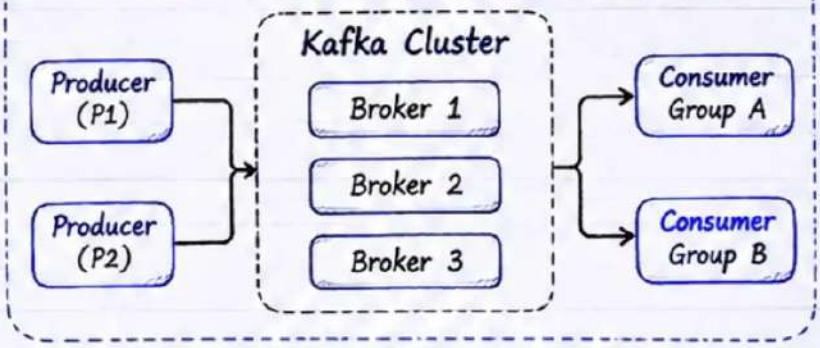
Insta ID: @aditechnova18

SUBTOPIC : Kafka Architecture & Core Components

1. Kafka Architecture – Overview

- Kafka follows a **distributed**, **scalable** and **fault-tolerant** architecture.
- It is a cluster of **brokers** that stores data (events) in **topics** and allows **producers** to publish and **consumers** to read the data.

Kafka High-Level Architecture



2. Core Components of Kafka

- **Producer** : Application that publishes (writes) data to a Kafka topic.
- **Consumer** : Application that reads (consumes) data from a topic.
- **Broker** : Kafka server that stores data and handles client requests.
- **Topic** : A logical channel (category) to which data is published.
- **Partition** : A topic is divided into multiple partitions for parallelism and scalability.
- **Offset** : A unique, sequential ID of a message within a partition.
- **Cluster** : A group of multiple Kafka brokers working together.

3. Component Details & Key Points

Component	Details / Key Points
Producer	• Sends (writes) records to a topic. • Can choose the partition (via key or round-robin).
Consumer	• Reads (consumes) records from a topic. • Can be part of a consumer group.
Broker	• Kafka server that stores data (logs). • Handles read/write - requests and replication.
Topic	• Logical stream of related events / messages. • Can have multiple partitions.
Partition	• Subdivision of a topic. • Enables parallel processing & high throughput.
Offset	• Position of a message in a partition. • Helps consumer track what has been read.
Cluster	• Collection of multiple brokers. • Provides scalability, high availability & fault tolerance.

4. Kafka Component Flow (Working)

1. **Producer** sends a record to a **Topic**.
2. **Topic** is split into **Partitions** and stored across **Brokers**.
3. **Consumers** (in a group) read records from the partitions.
4. Kafka maintains the **Offset** for each record read by a consumer.

★ Key Takeaways

- ☑ Kafka is a **distributed** event streaming platform.
- ☑ Data is organized in **topics** → **partitions** → **offsets**.
- ☑ Producers write and consumers read the data.
- ☑ Brokers store data and form a cluster.
- ☑ Partitions give **scalability** and **parallelism**.
- ☑ Consumer groups enable load balancing.

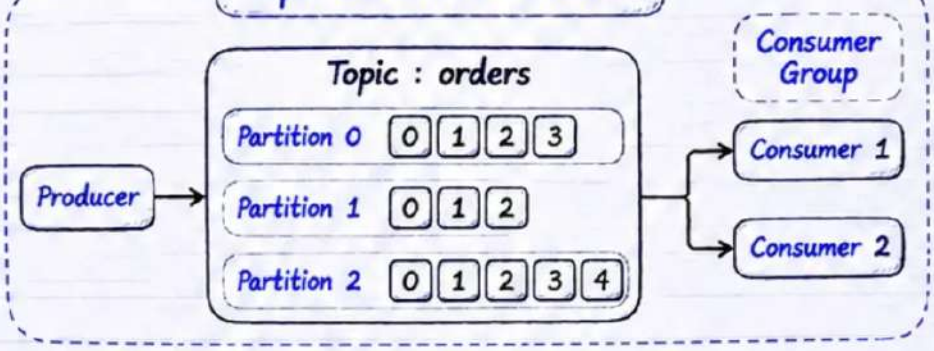
Insta ID: @aditechnova18

SUBTOPIC : Topics, Partitions & Offsets

1. Kafka Topic

- A **Topic** is a logical channel or category to which producers **publish** messages and consumers **subscribe** to read them.
- A topic can have multiple **partitions** for parallel processing and scalability.

Topic Structure in Kafka

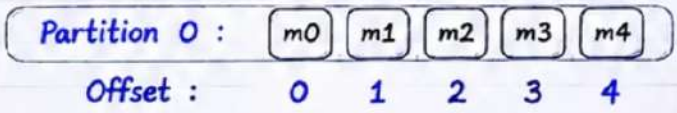


2. Kafka Partition

- A **Partition** is a division of a topic that holds a subset of its messages.
- Each partition is an **ordered**, **immutable** sequence of records.
- Partitions enable :
  - ☑ Parallel processing (multiple consumers).
  - ☑ Higher throughput.
  - ☑ Better scalability.

3. Offset

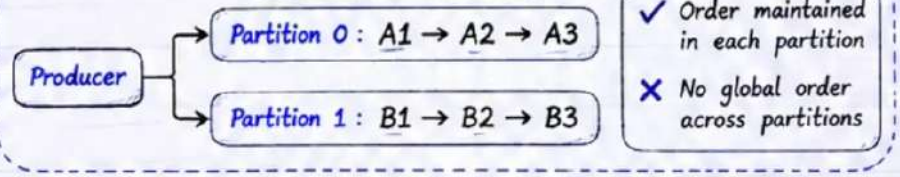
- An **Offset** is a unique, sequential ID assigned to each message within a partition.
- It represents the **position** of the message in that partition.
- Consumers use offsets to keep track of which messages they have already read.



4. Message Ordering

- Kafka guarantees message ordering **within a partition only**.
- No ordering guarantee across **different partitions** of the same topic.

Ordering Example



5. Replication (Basics)

- Each partition can have multiple copies called **replicas**.
- One replica acts as the **Leader** (handles read/write) and others as **Followers** (keep a copy).
- Improves **fault tolerance** and **high availability**.

6. Example : Partitioning & Offsets

- Topic : payment-events
- Partitions : 2
- Replication factor : 3
- New messages published : P1, P2, P3, P4

Message	Key	Partition (hash)	Offset
P1	user1	0	0
P2	user2	1	0
P3	user1	0	1
P4	user3	1	1

★ Key Takeaways

- ☑ Topic = logical stream of messages.
- ☑ Partition = unit of parallelism & scalability.
- ☑ Offset = unique position of a message.
- ☑ Replication = multiple copies for reliability.

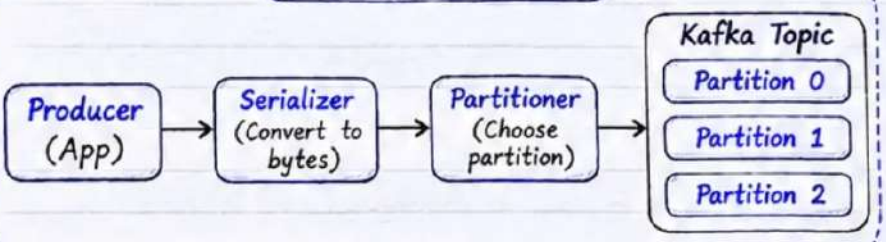
Insta ID: @aditechnova18

SUBTOPIC : Kafka Producers in Depth

1. What is a Producer ?

- A **Producer** is an application that sends (writes) records / messages to a **Kafka topic**.
- It decides which **partition** to send a message to and can configure **reliability**, **batching** and **compression**.

Producer Workflow



2. KafkaProducer API (Java)

- Kafka provides `KafkaProducer<K, V>` class to send messages to a topic.
- Key classes:
  - ☑ **ProducerConfig** – for configuration.
  - ☑ **KafkaProducer** – main producer class.
  - ☑ **ProducerRecord** – represents a message.

3. Producer Example (Java)

```
// Create producer and send a message
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer",
    "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer",
    "org.apache.kafka.common.serialization.StringSerializer");

KafkaProducer<String, String> producer =
        new KafkaProducer<>(props);
ProducerRecord<String, String> record =
        new ProducerRecord<>("orders", "order-1001");
producer.send(record);
producer.close();
```

4. Serialization

- Kafka sends data in the form of **bytes**.
- A **Serializer** converts the key and value to bytes before sending.
- Common serializers:
  - ☑ **StringSerializer** – for String data.
  - ☑ **IntegerSerializer** – for Integer data.
  - ☑ **JsonSerializer** – for JSON objects (e.g., POJO).

5. Acknowledgements (acks)

- **acks** controls how many **replicas** must acknowledge the write.

acks	Meaning	Reliability & Latency
0	No acknowledgement from broker.	Fastest, but may lose data.
1	Leader acknowledges (write successful).	Balanced (default), leader failure may cause data loss.
all (-1)	All in-sync replicas (ISR) acknowledge.	Highest reliability, comparatively higher latency.

6. Key-based Partitioning

- Kafka uses a **partitioner** to decide which partition to send a record to.
- Default : Hash of the key → same key goes to the same partition (ensures ordering per key).

`partition = hash(key) % numPartitions`

7. Batching & Compression

- **Batching** : Groups multiple records and sends them together (improves throughput).
- **Compression** : Compresses the batch to reduce network usage.
- Supported types : **none**, **gzip**, **snappy**, **lz4**, **zstd**.

★ Key Points

- ☑ Producer writes data to a topic.
- ☑ Data is serialized, batched and sent to a partition.
- ☑ **acks** controls the durability of the write.
- ☑ Key determines the partition (hash-based).

★ Interview Tip

- Default **acks** is **1**.
- Use **acks = all** for high reliability systems (e.g., payments, orders).
- Use compression and batching for better performance.

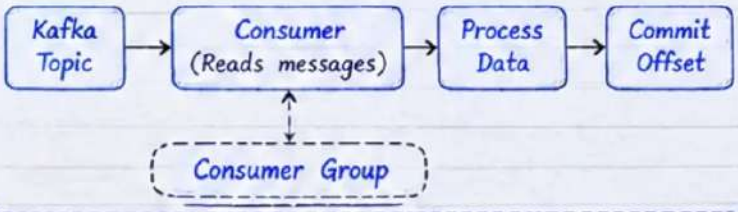
Insta ID: @aditechnova18

SUBTOPIC : Consumers & Consumer Groups

1. What is a Consumer?

- A Consumer reads messages from a Kafka topic and processes them.
- It maintains its own offset to keep track of the messages it has consumed.
- Consumers can belong to a consumer group for scalable and fault-tolerant processing.

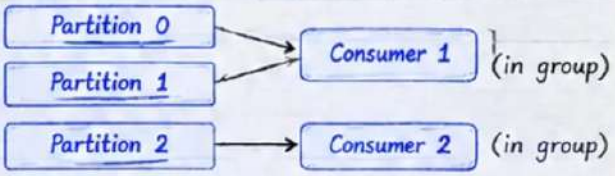
Consumer Workflow



2. Consumer Groups

- A consumer group is a group of consumers that work together to consume data from a topic.
- Each partition is consumed by only one consumer in a group at a time.
- If you have more consumers than partitions, some consumers will be idle.

Example: 3 Partitions, 2 Consumers



3. Group Coordination

- One consumer in the group acts as the group coordinator.
- It manages partition assignment and rebalancing.
- When a consumer joins or leaves, the group rebalance happens.

Rebalancing Triggers

- New consumer joins
- Consumer leaves
- Consumer failure
- Partition change

Rebalance Process

1. Detect change
2. Revoke partitions
3. Assign new partitions
4. Resume consumption

4. Consumer Offsets

- Offset = position of the next message to be consumed.
- Stored in Kafka (typically in __consumer_offsets topic).
- Can be managed automatically or manually.

Auto vs Manual Commit

Feature	Auto Commit	Manual Commit
Commit timing	Periodic (default)	After processing
Loss of data	Possible (on crash)	Less likely
Control	Less	More
Use case	Simple apps	At-least-once / exactly-once

5. Auto vs Manual Commit (in Java)

Auto Commit (default)

```
props.put(ConsumerConfig.
  ENABLE_AUTO_COMMIT_CONFIG,
  "true");
props.put(ConsumerConfig.
  AUTO_COMMIT_INTERVAL_MS_CONFIG,
  "1000");
```

Manual Commit

```
consumer.subscribe(Arrays.
  asList("orders"));
while (true) {
    ConsumerRecords<String,
        String> records =
        consumer.poll(Duration.
            ofMillis(1000));
    // process records
    consumer.commitSync();
}
```

6. Consumer Lag

- Lag = difference between the latest offset and the consumer's committed offset.
- High lag means consumer is falling behind.
- Can be monitored using tools like Burrow, Control Center, or Kafka Manager.

7. Rebalancing & Fault Tolerance

- Rebalancing ensures even partition distribution.
- If a consumer fails, partitions are reassigned.
- Kafka ensures fault tolerance by allowing other consumers to take over.

★ Key Takeaways

- ☑ Consumers read data from topics.
- ☑ Consumer groups enable parallelism and scalability.
- ☑ Offsets track message position.
- ☑ Rebalancing happens on group changes.
- ☑ Manual commit gives more control.
- ☑ Monitor consumer lag for health check.

★ Interview Tip

- Difference between auto and manual commit.
- What happens during rebalance?
- How to handle consumer lag?
- Can a consumer read from multiple topics?

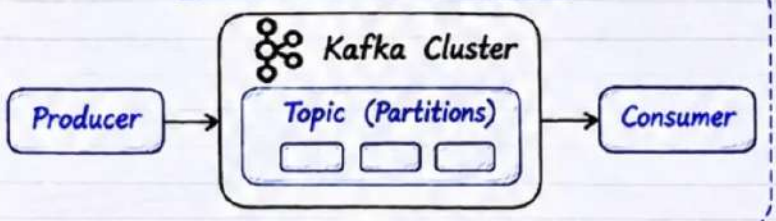
Insta ID: @aditechnova18

SUBTOPIC : Kafka Delivery Semantics

1. What are Delivery Semantics ?

- Delivery semantics define the guarantee about how many times a message will be delivered to the consumer.
- Kafka provides three delivery guarantees :
  - ☑ At-most-once
  - ☑ At-least-once
  - ☑ Exactly-once

End-to-End Message Flow



2. At-most-once Delivery

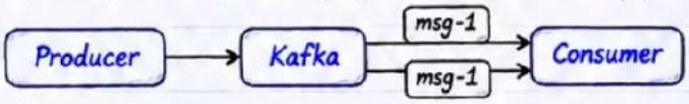
- Message is delivered **at most** one time.
- No retry if delivery fails (message may be lost).
- **Use case:** Logging, metrics, non-critical data.



(Message lost, no retry)

3. At-least-once Delivery

- Message is delivered **one** or **more** times.
- Producer retries until it gets an acknowledgement.
- No message loss, but duplicates can occur.
- **Use case:** Order events, payment events, data pipeline.

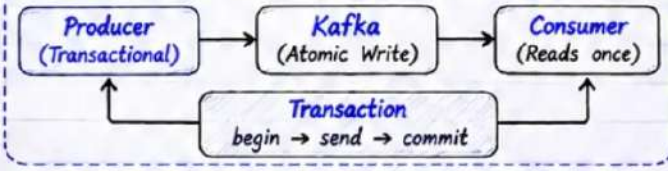


(Possible duplicate)

4. Exactly-once Delivery (EOS)

- Message is delivered **exactly once**.
- Achieved using **idempotent producer** and Kafka **transactions**.
- Guarantees no **loss** and no duplicates (end-to-end exactly once).
- **Use case:** Financial transactions, inventory updates, critical business data.

Exactly-once (with Transactions)



5. Idempotent Producer

- Ensures the same message is not written multiple times to a partition, even if retried.
- Enable using : `enable.idempotence = true`
- Internally uses Producer ID (PID) and sequence numbers.

6. Transactions in Kafka

- Allows **atomic** writes across multiple partitions/topics.
- Consists of: begin → send → commit / abort.
- Enables exactly-once processing (EOS).

7. Comparison of Delivery Semantics

Feature	At-most-once	At-least-once	Exactly-once
Delivery count	0 or 1	≥ 1	= 1
Message loss	Possible	No	No
Duplicates	No	Possible	No
Complexity	Low	Medium	High
Use case	Logs, metrics	Orders, events	Payments, financial data

8. Key Configuration (Examples)

Configuration	Purpose
acks = 0	At-most-once (no acknowledgement)
acks = 1	Leader acknowledgement (default)
acks = all	Stronger durability (all ISR replicas)
enable.idempotence = true	Enables idempotent producer
transactional.id = "app-txn"	Enables transactions for EOS

★ Key Takeaways

- ☑ Choose delivery semantics based on business need.
- ☑ At-most-once → Fast, may lose messages.
- ☑ At-least-once → No loss, but duplicates possible.
- ☑ Exactly-once → Strongest, using idempotence + transactions.
- ☑ Use acks=all + idempotence + transactions for critical systems.

★ Interview Questions

- What are Kafka's delivery guarantees?
- How does Kafka achieve exactly-once delivery?
- What is an idempotent producer?
- How do Kafka transactions work?
- When would you choose at-least-once over exactly-once?
- How to handle duplicate messages in Kafka?

Insta ID: @aditechnova18

SUBTOPIC : Replication, Leaders & Fault Tolerance

1. Why Replication in Kafka ?

- Replication keeps multiple copies of data (partitions) across different brokers.
- It provides :
  - ☑ **Fault Tolerance** (no data loss).
  - ☑ **High Availability** (service keeps running).
  - ☑ **Durability** (data is safe even if a broker fails).

2. Leader and Follower Replicas

- Each partition has multiple replicas.
- One replica is the **Leader** (handles all read & write).
- Other replicas are **Followers** (keep a copy of data from leader).
- If leader fails, one of the followers is promoted to leader (**leader election**).

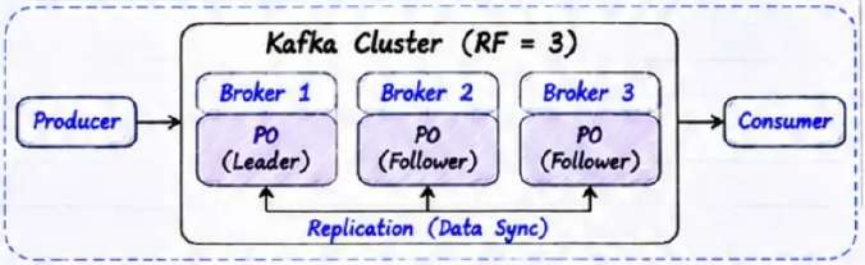
3. Replication Factor (RF)

- Replication factor = number of copies of each partition (leader + followers).
- **RF = 1** → No replication (not fault tolerant).
- **RF = 2** → 1 leader + 1 follower.
- **RF = 3** → 1 leader + 2 followers (recommended).

4. In-Sync Replicas (ISR)

- **ISR** = set of replicas that are fully caught up with the leader.
- Only replicas in ISR are **eligible** to become leader.
- Keeps data consistent and reduces data loss.

5. Kafka Replication Architecture



6. Leader Election

- When a leader broker fails, Kafka elects a new leader from the ISR.
- Done automatically by the **Kafka controller** (in **KRaft** or **ZooKeeper**-based clusters).
- Ensures continuous availability with minimal downtime.

7. min.insync.replicas (Important)

- A broker config that defines the **minimum** number of **ISR** replicas required to acknowledge a write.
- Works with producer **acks = all**.
- If ISR < min.insync.replicas → **write is rejected** (ensures strong durability).

8. Putting It All Together (Example)

RF = 3, min.insync.replicas = 2, acks = all

Scenario	Result
All 3 replicas (ISR = 3)	Write succeeds (ack from ≥ 2)
One follower down (ISR = 2)	Write succeeds (ack from ≥ 2)
Two brokers down (ISR = 1)	Write fails (ISR < 2)
Leader down	New leader elected from ISR

9. Fault Tolerance in Kafka

- **Broker failure** → automatic leader election.
- **Data loss prevention** → replication + ISR + min.insync.replicas.
- **Consumer failure** → consumer group rebalances and reassigns partitions.
- **Network issues** → replicas keep syncing when connection recovers.

10. Replication Configuration (Sample)

```
# server.properties (broker config)
num.replication.factor=3          # default replication factor
min.insync.replicas=2             # min ISR for successful write
unclean.leader.election.enables=false # avoid data loss
acks=all                          # producer waits for all ISRs
```

★ Key Takeaways

- ☑ Replication = multiple copies for safety.
- ☑ Leader handles reads/writes, followers replicate.
- ☑ ISR = up-to-date replicas.
- ☑ min.insync.replicas + acks = all ⇒ strong reliability.
- ☑ Leader election keeps the system highly available.

★ Interview Questions

- What is the role of replication in Kafka?
- What is ISR in Kafka?
- How does leader election work?
- Explain the min.insync.replicas configuration.
- What happens if the leader and one follower go down?
- How does Kafka ensure fault tolerance and high availability?

Insta ID: @aditechnova18

SUBTOPIC : Kafka Installation & CLI Commands

1. Prerequisites & Setup Options

- **Requirements :**
  - ☑ Java (JDK 11 or 17)
  - ☑ Kafka binaries (3.x or later)
- **Setup options :**
  - Local setup (single node)
  - Docker setup (recommended)
  - Production cluster (multi-broker)

2. Kafka Versions & KRaft Mode

- **KRaft (Kafka Raft) :** Newer mode that removes ZooKeeper and uses Kafka's own Raft consensus for metadata management.
- **ZooKeeper mode :** Older mode, still supported in some versions.
- For new projects, use KRaft (simpler & lighter).

3. Local Installation (Steps)

① Download Kafka from :

```
https://kafka.apache.org/downloads
```

② Extract the tar :

```
tar -xzf kafka_3.6.0.tgz
```

③ Move to Kafka directory :

```
cd kafka_3.6.0
```

④ (KRaft) Generate cluster ID :

```
bin/kafka-storage.sh random-uuid
```

⑤ Format storage (KRaft) :

```
bin/kafka-storage.sh format \
 --cluster-id <CLUSTER_ID> \
 --config config/kraft/server.properties
```

4. Start Kafka (KRaft Mode)

- Start the broker (single node) :

```
bin/kafka-server-start.sh \
 config/kraft/server.properties
```

- Stop the broker :

```
bin/kafka-server-stop.sh
```

★ Note

- In KRaft mode, no ZooKeeper is required.
- For older versions (use ZooKeeper) :
  ① Start ZooKeeper
  ② Start Kafka broker

5. Kafka Directory Structure

```
kafka_3.6.0/
├── bin/      → Scripts (start/stop, CLI tools)
├── config/   → Configuration files (server.properties)
├── logs/     → Topic data (message logs)
└── libs/     → Kafka JARs
```

6. Essential Kafka CLI Commands

Task	Command (KRaft or ZooKeeper)
Create a topic	bin/kafka-topics.sh --create --topic orders --partitions 3 --replication-factor 1
List all topics	bin/kafka-topics.sh --list
Describe a topic	bin/kafka-topics.sh --describe --topic orders
Delete a topic	bin/kafka-topics.sh --delete --topic orders
Produce messages	bin/kafka-console-producer.sh --topic orders
Consume messages	bin/kafka-console-consumer.sh --topic orders --from-beginning
List consumer groups	bin/kafka-consumer-groups.sh --list
Describe consumer group	bin/kafka-consumer-groups.sh --describe --group my-group
Reset consumer offset	bin/kafka-consumer-groups.sh --group my-group --topic orders \ --reset-offsets --to-earliest --execute

★ Common Flags

- `--topic` : Topic name
- `--partitions` : No. of partitions
- `--replication-factor` : No. of replicas
- `--from-beginning` : Read all existing data
- `--group` : Consumer group name
- `--to-earliest` / `--to-latest` : Reset offsets

7. Testing the Setup

- ☑ Start Kafka broker
- ☑ Create a topic (e.g., test-topic)
- ☑ Produce a few messages
- ☑ Consume the messages (in another terminal)
- ☑ Verify offsets and consumer group

★ Key Takeaways

- ☑ Use KRaft mode for new setups.
- ☑ Kafka provides powerful CLI tools for topic, producer & consumer ops.
- ☑ You can easily create, produce and consume messages from the terminal.
- ☑ Docker is a quick alternative for setup.

★ Interview Questions

- What is KRaft in Kafka?
- How do you create and describe a topic?
- How to reset a consumer group offset?
- Difference between KRaft and ZooKeeper mode?
- What are the most important Kafka CLI commands?

Insta ID: @aditechnova18

SUBTOPIC : Kafka with Java & Spring Boot

1. Using Kafka with Java (Overview)

- Kafka provides official Java client libraries :
  - ☑ kafka-clients (producer & consumer)
  - ☑ kafka-streams (stream processing)
  - ☑ kafka-connect (data integration)
- Add dependency (Maven) :

```
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>3.6.0</version>
</dependency>
```

2. Java Kafka Producer (Example)

```
import org.apache.kafka.clients.producer.*;
import java.util.Properties;
public class SimpleProducer {
  public static void main(String[] args) {
     Properties props = new Properties();
     props.put("bootstrap.servers", "localhost:9092");
     props.put("key.serializer",
          "org.apache.kafka.common.serialization.StringSerializer");
     props.put("value.serializer",
          "org.apache.kafka.common.serialization.StringSerializer");
     KafkaProducer<String, String> producer = new KafkaProducer<>(props);
     ProducerRecord<String, String> record =
        new ProducerRecord<>("orders", "Order placed!");
     producer.send(record);
     producer.close();
  }
}
```

3. Java Kafka Consumer (Example)

```
import org.apache.kafka.clients.consumer.*;
import java.util.*;
public class SimpleConsumer {
  public static void main(String[] args) {
     Properties props = new Properties();
     props.put("bootstrap.servers", "localhost:9092");
     props.put("group.id", "order-group");
     props.put("key.deserializer",
        "org.apache.kafka.common.serialization.StringDeserializer");
     props.put("value.deserializer",
        "org.apache.kafka.common.serialization.StringDeserializer");
     KafkaConsumer<String, String> consumer =
        new KafkaConsumer<>(props);
     consumer.subscribe(Arrays.asList("orders"));
     while (true) {
        ConsumerRecords<String, String> records =
           consumer.poll(Duration.ofSeconds(2));
        for (ConsumerRecord<String, String> r : records) {
           System.out.println("Received : " + r.value());
        }
     }
  }
}
```

4. Spring Boot + Kafka (Integration)

- Spring Boot provides easy integration using spring-kafka.
- Add dependency (pom.xml) :

```
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
```

5. Spring Boot Configuration

```
# application.properties
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=order-group
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.producer.key-serializer=
   org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=
   org.apache.kafka.common.serialization.StringSerializer
```

6. Spring Boot Producer & Consumer

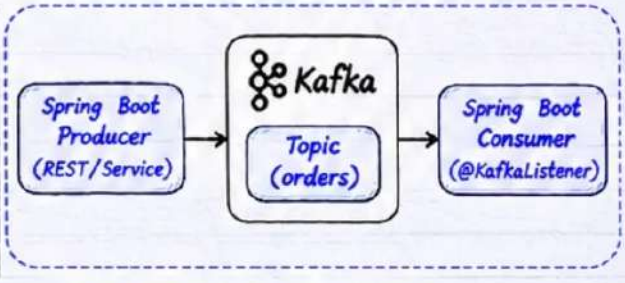
Producer (Service)

```
@Service
public class OrderProducer {
   @Autowired
   private KafkaTemplate<String,
         String> kafkaTemplate;
   public void sendOrder(String msg){
      kafkaTemplate.send("orders", msg);
   }
}
```

Consumer (Listener)

```
@Component
public class OrderConsumer {
   @KafkaListener(topics = "orders",
                  groupId = "order-group")
   public void consume(String msg) {
      System.out.println("Order Received :"
                         + msg);
   }
}
```

7. Flow Diagram (Spring Boot)



★ Key Takeaways

- ☑ Use kafka-clients for pure Java, spring-kafka for Spring Boot.
- ☑ Configure serializers, bootstrap servers and group-id.
- ☑ Use @KafkaListener for consuming messages easily.
- ☑ KafkaTemplate is the high-level API for producing messages.
- ☑ Spring Boot makes Kafka integration production-ready.

★ Interview Questions

- How do you integrate Kafka with Spring Boot?
- What is KafkaTemplate and @KafkaListener?.
- How to configure producer and consumer in application.properties?
- How do you send and consume messages in Spring Boot?
- What is the role of group.id in Kafka?

Insta ID: @aditechnova18

SUBTOPIC : Kafka Performance & Optimization

1. Why Performance Tuning ?

- Kafka can handle high throughput, but proper configuration is needed to achieve:
  - ☑ Higher throughput (more msgs/sec)
  - ☑ Lower latency (faster delivery).
  - ☑ Better resource utilization (CPU, disk, network)
  - ☑ Stability under heavy load

2. Key Performance Factors

- Producer configuration (batching, compression, acks).
- Consumer configuration (poll size, concurrency).
- Broker & topic settings (partitions, replication, retention).
- Hardware resources (CPU, RAM, disk I/O, network).
- Data size and message format (e.g., Avro, Protobuf).

3. Producer Optimization

Configuration	Default	Tuning Tip
batch.size	16 KB	Increase (e.g., 64 KB) for higher throughput
linger.ms	0 ms	Set (5–50 ms) to allow batching of more messages
compression.type	none	snappy (recommended), lz4 or zstd for better size
acks	1	Use acks=1 or all (for durability)
retries	0	Increase (e.g., 3) for reliability
buffer.memory	32 MB	Increase if producer sends large volume

4. Consumer Optimization

Configuration	Default	Tuning Tip
fetch.min.bytes	1 B	Increase (e.g., 1 MB) to fetch larger batches
fetch.max.wait.ms	500 ms	Set (100–1000 ms) to wait for more data
max.poll.records	500	Increase (e.g., 1000–5000) for batch processing
enable.auto.commit	true	Use false and commit manually for better control
max.poll.interval.ms	5 min	Increase for long processing tasks

5. Broker & Topic Level Tuning

Setting	Purpose / Tuning
num.partitions	Increase for higher parallelism (e.g., 6, 12, 24).
replication.factor	Use 3 (balance between durability and performance).
log.retention.hours	How long to keep data (e.g., 168 = 7 days).
log.segment.bytes	Size of log file (e.g., 1 GB).
min.insync.replicas	Set to 2 (with RF=3) for safety.
log.cleanup.policy	delete (default) or compact (for key-based latest value).

6. Batching & Compression

- Batching : Groups multiple messages and sends them together → higher throughput.
- Compression : Reduces message size → less network & disk usage.
- Common compression types :

Type	Compression Ratio	Speed	Use Case
snappy	Medium	Fast	General purpose (Recommended)
lz4	High	Very Fast	High throughput
zstd	Very High	Medium	Best compression, more CPU usage
gzip	High	Slow	When size is priority

7. Scalability Best Practices

- Use multiple partitions to parallelize consumers.
- Run multiple consumer instances in a consumer group.
- Scale brokers horizontally (add more brokers).
- Use appropriate replication factor (e.g., 3).
- Monitor key metrics (throughput, lag, disk, CPU, network).

8. Monitoring & Key Metrics

Metric	Meaning
Throughput (msg/sec)	No. of messages produced/consumed per second.
Consumer Lag	How far a consumer is behind the latest offset.
Partition Count	No. of partitions per topic.
Broker CPU / Disk I/O	Resource usage of Kafka broker.
Network In/Out	Data transfer rate (MB/s).

★ Key Takeaways

- ☑ Tune producer (batching, compression, acks) for speed.
- ☑ Tune consumer (poll size, fetch settings) for efficiency.
- ☑ Use right partition & replication factor.
- ☑ Monitor metrics like throughput and consumer lag.
- ☑ Scale brokers and consumers as the data grows.

★ Interview Questions

- How can we improve Kafka producer performance?
- Which producer/consumer configurations impact throughput?
- What is the role of batching and compression in Kafka?
- How do you handle consumer lag?
- How do you scale Kafka for a high-traffic system?

Insta ID: @aditechnova18

SUBTOPIC : Kafka Advanced Concepts

1. Kafka Streams (Stream Processing)

- A client library to build real-time stream processing applications on Kafka.
- Allows data transformation, aggregation, join, and windowing.



- Use case : Real-time analytics, fraud detection, recommendation system.

2. Kafka Connect

- A framework to easily integrate Kafka with external systems (databases, cloud, file systems, etc.).
- Two types :
  - ☑ Source Connector : Pulls data from external system into Kafka.
  - ☑ Sink Connector : Pushes data from Kafka to external system.
- Examples : JDBC, MySQL, Elasticsearch, S3, HDFS.

3. Schema Registry & Data Serialization

- Manages and stores the schema (structure) of data (e.g., Avro, JSON, Protobuf).
- Ensures data compatibility between producers and consumers.
- Common serialization formats :

Format	Pros	Use Case
Avro	Compact, with schema (Registry)	Most used in Kafka
JSON	Human readable, simple	Simple & small data
Protobuf	Fast, efficient, strongly typed	High performance apps

4. Kafka Security

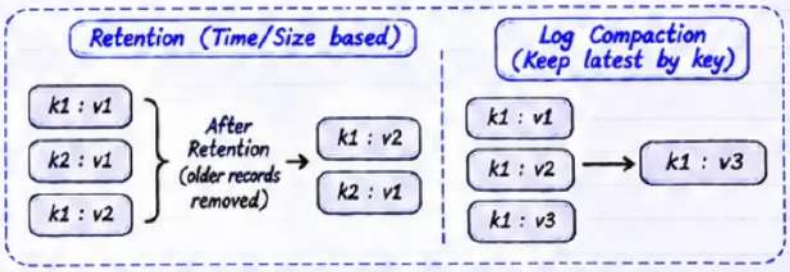
- Kafka provides multiple security mechanisms :
  - ☑ Authentication : Verifies the identity (SASL, Kerberos).
  - ☑ Authorization : Controls access using ACLs (Access Control Lists).
  - ☑ Encryption : Secures data in transit using SSL/TLS.

Common Security Config (example)

```
security.protocol = SASL_SSL
sasl.mechanism    = PLAIN       # or SCRAM, GSSAPI
sasl.jaas.config  = "org.apache.kafka.common.security.\
        plain.PlainLoginModule required \
        username=\"user\" password=\"pass\";"
```

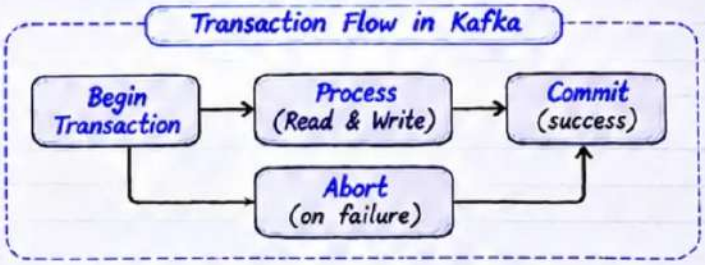
5. Kafka Retention & Log Compaction

- Retention : Defines how long data is kept in a topic.
  - ☑ Time-based : e.g., retention.ms = 7 days
  - ☑ Size-based : e.g., retention.bytes = 1GB
- Log Compaction : Keeps only the latest value for a given key (removes old duplicates).
- Use case : Maintaining the latest state (e.g., user profile, product catalog).



6. Exactly-Once Processing (EOS) in Depth

- Guarantees that each message is processed and written exactly once.
- Uses two key features :
  - ☑ Idempotent Producer (enable.idempotence = true)
  - ☑ Transactions (begin, commit, abort)
- Useful for : building reliable, duplicate-free data pipelines (e.g., ETL, real-time processing).



7. Kafka vs Other Streaming Tools

Feature	Kafka	RabbitMQ	Pulsar
Model	Distributed Log	Message Queue	Pub-Sub + Queue
Throughput	Very High	Medium	High
Durability	High	Medium	High
Stream Processing	Yes (Kafka Streams)	No	Yes
Use Case	Real-time, Big Data	Task Queue, RPC	Real-time, Multi-tenant

★ Key Takeaways

- ☑ Kafka Streams → real-time data processing.
- ☑ Kafka Connect → easy integration with external systems.
- ☑ Schema Registry → manages data schema & compatibility.
- ☑ Security → authentication, authorization, encryption.
- ☑ Retention & Compaction → control data storage.
- ☑ EOS → reliable, exactly-once data pipelines.

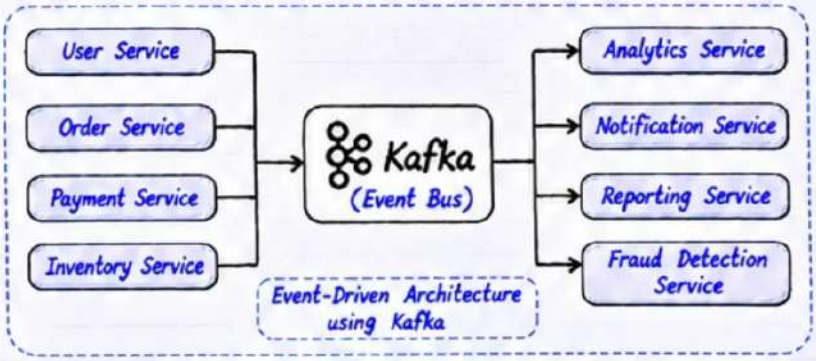
Insta ID: @aditechnova18

SUBTOPIC : Real-World Distributed System Design with Kafka

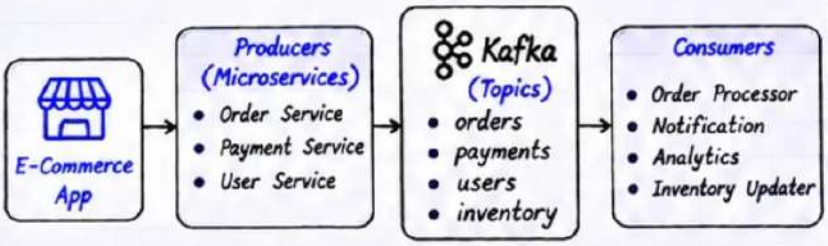
1. Real-World Use Cases of Kafka

- **Event-driven architecture :** Loose coupling between microservices.
- **Log aggregation :** Collect and process logs from multiple services.
- **Real-time analytics :** Stream and analyze data (clicks, transactions, etc.).
- **Fraud detection :** Process events in real-time to detect suspicious activities.
- **Recommendation system :** Build real-time user activity pipelines.

2. High Level Architecture (Event-Driven System)



3. Example : E-Commerce System



4. End-to-End Data Flow (Example)

1. User places an order in the E-commerce app.
2. Order Service (producer) sends an event to Kafka topic : `orders`.
3. Multiple consumers read the event :
   - Order Processor → process & update order
   - Notification Service → send order confirmation
   - Analytics Service → update real-time dashboards.
4. All services work asynchronously and independently.

5. Design Considerations

Aspect	Considerations / Best Choice
Partitioning	Based on key (e.g., userId / orderId) for parallelism and ordering.
Replication	Use RF = 3 for high availability and durability.
Delivery Semantics	At-least-once (default), Exactly-once (if needed).
Retention	Time-based (e.g., 7 days) or size-based (e.g., 10 GB).
Scalability	Add partitions and brokers as traffic grows.
Security	Enable SASL/SSL, ACLs for access control.
Monitoring	Use tools like Prometheus, Grafana, Kafka UI.

6. Sample Topic Design (E-Commerce)

Topic	Key	Partitions	Retention	Use Case
orders	orderId	6	7 days	New order events
payments	paymentId	6	7 days	Payment status events
users	userId	4	30 days	User profile updates
inventory	productId	6	7 days	Stock update events

7. Advantages in Distributed Systems

- ☑ Decouples services (producers and consumers work independently).
- ☑ High throughput & low latency (handles large event volume).
- ☑ Scalable & fault-tolerant (via partitions & replication).
- ☑ Supports real-time and batch processing together.
- ☑ Acts as a central event bus for the entire system.

8. Best Practices

- ☑ Use meaningful topic names (e.g., orders, payments).
- ☑ Choose the right partition key (e.g., userId) for ordering and load distribution.
- ☑ Keep a suitable replication factor (≥ 3).
- ☑ Monitor lag, throughput and consumer health.
- ☑ Use Schema Registry for schema evolution.
- ☑ Apply security (SSL/SASL, ACLs) in production.

★ Key Takeaways

- ☑ Kafka fits well in event-driven, real-time, distributed systems.
- ☑ Design around data flow, partitioning, replication and retention.
- ☑ Ensure scalability, reliability and security for production use.

★ Interview Questions

- How would you design an event-driven e-commerce system using Kafka?
- How do you choose the number of partitions for a topic?
- How does Kafka ensure high availability and fault tolerance?
- What are the key design considerations in a production Kafka setup?
- How would you handle retries and failures in a Kafka-based system?

Insta ID: @aditechnova18

SUBTOPIC : FAANG-Level Kafka Interview Questions

1. Core Concepts & Basics

Q1. What is Apache Kafka ?
Ans : A distributed, fault-tolerant, high-throughput event streaming platform used for real-time data pipelines and stream processing.

Q2. What are the main components of Kafka ?
Ans : Producer, Consumer, Topic, Partition, Broker, Zookeeper (or KRaft), and Consumer Group.

Q3. What is a topic and a partition ?
Ans : Topic is a logical channel to store records. Partition is a smaller, ordered and immutable log of a topic which enables parallelism.

2. Producers & Consumers

Q4. How does a Kafka producer work ?
Ans : It serializes records, selects a partition (via partitioner), and sends data to the broker with acknowledgement (acks).

Q5. What are consumer groups ?
Ans : A group of consumers that share the load by reading different partitions of the same topic.

Q6. How are offsets managed in Kafka ?
Ans : Kafka stores consumer offsets in an internal topic __consumer_offsets, and consumers can commit offsets automatically or manually.

3. Delivery & Reliability

Q7. What are Kafka's delivery semantics ?
Ans :
- At-most-once : Message may be lost.
- At-least-once : Message may be reprocessed.
- Exactly-once : Message is processed only once (using idempotent producer + transactions).

Q8. How do you achieve exactly-once semantics ?
Ans : By enabling idempotence (enable.idempotence=true) and using transactions (transactional.id).

4. Replication & Fault Tolerance

Q9. What is replication factor and ISR ?
Ans : Replication factor (RF) = Number of copies of a partition. ISR (In-Sync Replicas) = replicas that are up-to-date with the leader.

Q10. What happens if a broker fails ?
Ans : A follower from ISR is elected as the new leader, and the system continues without data loss.

Q11. What is min.insync.replicas ?
Ans : Minimum number of in-sync replicas required to accept a write, ensuring data durability.

5. Performance & Scalability

Q12. How would you increase Kafka throughput ?
Ans :
- Increase partitions
- Use batching (batch.size, linger.ms)
- Enable compression (snappy, lz4)
- Tune producer/consumer configurations.

Q13. How does Kafka scale ?
Ans : Horizontally, by adding more brokers and partitions. Producers and consumers can work in parallel.

6. Advanced Features

Q14. What is Kafka Streams ?
Ans : A client library for building real-time stream processing applications (filter, aggregate, join, etc.).

Q15. What is Kafka Connect ?
Ans : A framework to integrate Kafka with external systems using Source and Sink connectors.

Q16. What is Schema Registry ?
Ans : A service to manage and enforce schemas (Avro, JSON, Protobuf) for serialized data.

7. Operations & Production

Q17. How do you monitor a Kafka cluster ?
Ans : Use metrics like throughput, consumer lag, broker health, disk usage, etc., with tools such as Prometheus + Grafana.

Q18. How do you handle data retention in Kafka ?
Ans : Configure retention by time (retention.ms) or size (retention.bytes) at the topic level.

8. Scenario / Design Questions

Q19. Design a real-time notification system using Kafka.
Ans : Use microservices (publisher → Kafka → consumer), separate topics (e.g., notifications), and multiple consumers for email, SMS and in-app notifications.

Q20. How would you design an e-commerce event pipeline ?
Ans : Publish events (order-created, payment-success, etc.) to Kafka topics; multiple consumer services process them (inventory update, notification, analytics) independently.

★ Key Takeaways

- ☑ Understand core concepts (topics, partitions, offsets).
- ☑ Know delivery semantics and how to achieve EOS.
- ☑ Be clear on replication, ISR, and fault tolerance.
- ☑ Be able to tune for performance and scalability.
- ☑ Prepare for real-world design and use-case questions.

★ Common FAANG Follow-ups

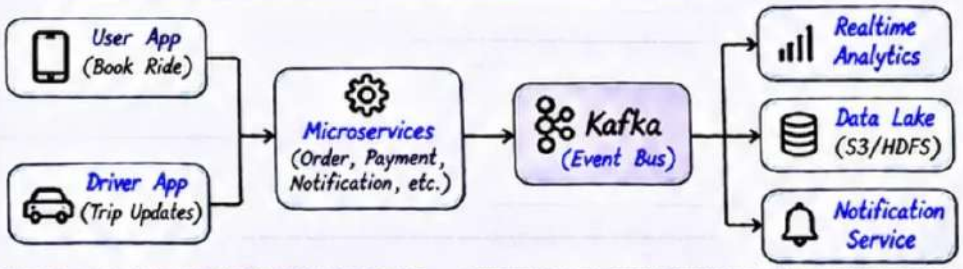
- Difference between Kafka and other messaging systems (e.g., RabbitMQ, Pulsar).
- How to handle backpressure and consumer lag ?
- How to ensure message ordering ?
- How to choose the right number of partitions ?
- Real-world case studies (Uber, Netflix, LinkedIn, etc.).

Insta ID: @aditechnova18

SUBTOPIC : System Design, Best Practices & Final Revision

1. System Design : Real-World Example

Use Case : Real-time Ride Booking System (Uber-like)



Event Flow :

1. User/Driver actions → publish events (e.g., ride_created, ride_updated).
2. Kafka topics store events (partitioned & replicated).
3. Multiple consumers : order service, analytics, notification.

Topic Design (Example)

Topic	Partitions	Replication	Retention
ride-events	12	3	7 days
payment-events	6	3	30 days
notification-events	6	3	3 days

2. Production Best Practices

- ☑ Use meaningful **topic names** (e.g., user-events, order-events).
- ☑ Choose proper **partition count** (based on throughput & parallelism).
- ☑ Keep **replication factor ≥ 3** for HA.
- ☑ Enable **acks = all** for durability.
- ☑ Use **compression** (snappy or **lz4**).
- ☑ Tune **batch.size** and **linger.ms** for high throughput.
- ☑ Monitor with **Prometheus** + **Grafana**.
- ☑ Apply **security** (SASL/SSL, ACLs).
- ☑ Define appropriate **retention** & log compaction policies.
- ☑ Test **rebalancing** and failure cases.

3. Common Issues & Troubleshooting

Issue / Error	Possible Cause	Fix / Solution
Consumer lag high	Slow processing / less consumers	Increase consumers, check processing time.
Rebalancing too often	Consumer instability (heartbeats)	Increase session.timeout.ms & ensure stable consumers.
Message not delivered	acks not set / broker down	Use acks = all and check broker health.
Producer timeout	Large batch / network latency	Tune batch.size, linger.ms and request.timeout.ms.
Data loss (on failure)	Replication factor = 1 or ack = 0	Use RF ≥ 3 and acks = all.

4. Useful Kafka Configuration (Quick Ref)

Property	Recommended Value	Purpose
acks	all	Ensures durability (no data loss).
retries	3 – 10	Retry on transient failures.
batch.size	64 KB (or 32 KB)	Batches messages for better throughput.
linger.ms	5 – 50 ms	Wait time to form a batch.
compression.type	snappy / lz4	Reduce message size & network usage.
replication.factor	≥ 3	High availability & fault tolerance.
min.insync.replicas	2	Min. replicas to accept a write.
retention.ms	7 days (example)	How long to keep data in a topic.

5. Kafka vs Other MQs (Quick Comparison)

Feature	Kafka	RabbitMQ	Pulsar
Model	Distributed log	Message queue	Pub-Sub + Queue
Throughput	Very High	Medium	High
Durability	High	Medium	High
Ordering	Per partition	Per queue	Per partition
Use Case	Real-time, Big Data	Task processing	Multi-tenant, Streaming

6. Interview Final Revision (One-Liners)

- **Kafka** = distributed, fault-tolerant, event streaming platform.
- **Topic** = logical channel; **Partition** = unit of parallelism.
- **Producer** = publishes messages; **Consumer** = reads messages.
- **Offsets** = position of a consumer in a partition.
- **Replication** = copies data to multiple brokers (RF ≥ 3).
- **Delivery semantics** = At-most-once, At-least-once, Exactly-once.
- **Kafka Streams** = real-time stream processing (filter, join, aggregate).
- **Kafka Connect** = easy integration with external systems (Source/Sink).
- **Schema Registry** = manage and evolve message schemas (Avro/JSON).
- **EOS** = Exactly-Once Semantics (idempotent producer + transactions).

★ Final Takeaways

- ☑ Kafka is best for high-throughput, real-time and event-driven systems.
- ☑ Understand core building blocks, configurations and trade-offs.
- ☑ Focus on delivery guarantees, replication, and scalability.
- ☑ Know real-world use cases and design patterns.
- ☑ Be ready with configuration, troubleshooting and LLD/HLD.

★ Quick Revision Checklist

- ☑ Kafka Architecture & Components
- ☑ Topics, Partitions & Offsets
- ☑ Producers & Consumers
- ☑ Consumer Groups & Rebalancing
- ☑ Replication & Fault Tolerance
- ☑ Delivery Semantics & EOS
- ☑ Kafka Streams & Connect
- ☑ Schema Registry & Serialization
- ☑ Performance & Tuning
- ☑ Real-world Design & Best Practices

Thank You!

For being a part of this journey

kafka

Build • Stream • Scale

This notebook is a complete guide to Apache Kafka, designed to help you understand, learn and grow from the basics to advanced concepts.

If you found these notes helpful, it means a lot! Your support, trust and curiosity keep me motivated to create more such resources.

Better Notes = Better Concepts = Better You

Quick Recap – What You Learned

- ☑ Kafka fundamentals & architecture
- ☑ Producers & consumers
- ☑ Delivery semantics & replication
- ☑ Kafka with Java & Spring Boot
- ☑ Advanced Kafka concepts
- ☑ Distributed system design
- ☑ FAANG interview preparation
- ☑ Production best practices

"Small steps every day lead to big results."



Keep learning,
Keep building,
Keep growing...

★ Special Thanks

- To you — for trusting me and being here. ♥
- To the Kafka community — for open source and amazing resources.
- To the developers, engineers and creators who share knowledge.
- And to myself — for not giving up.

Because every question you asked, brought me one step closer to understanding and building.

Keep Exploring
Keep Coding
Keep Dreaming



With best wishes,
Aditya Kumar
@aditechnova18

See you in the next notes! →

★★★